

Department of Computer Science

CS403NOD – Node.js

Lab Assignment – 06

Working With The File System (fs) Module

Semester: BCA VII

Lab No.: 06

Objective

After completing this lab, you should be able to:

- Read files using both the asynchronous and synchronous fs methods.
- Understand the practical difference this makes to program execution.
- Write, overwrite, and append content to files.
- Delete files using the fs module.
- Use the error-first callback pattern correctly in file operations.
- Rewrite callback-based fs code using async/await.
- Build a small working command-line application using file storage.

Tasks

Task 1 – Project Setup

Create the following folder structure.

```
NodeJS-Lab/  
└── Lab-06/  
    ├── sample.txt  
    ├── read-async.js  
    ├── read-sync.js  
    ├── write-file.js  
    ├── append-file.js  
    ├── delete-file.js  
    ├── async-await-version.js  
    ├── add-note.js  
    ├── read-notes.js  
    ├── reflection-notes.txt  
    └── README.md
```

Create sample.txt and type 2-3 lines of any text into it — this is the file your programs will read from in Tasks 2, 3, and 7.

Task 2 – Reading A File Asynchronously

Create

read-async.js

```
const fs = require('fs');
```

```
fs.readFile('sample.txt', 'utf8', (err, data) => {  
  if (err) {  
    console.error('Error reading file:', err.message);  
    return;  
  }  
  console.log(data);  
});
```

```
console.log('This line runs BEFORE the file content is printed.');
```

Run it and note the ORDER the two console.log lines appear in — write this down, you'll need it for Task 9.

Task 3 – Reading A File Synchronously (And Comparing)

Create

read-sync.js

```
const fs = require('fs');
```

```
const data = fs.readFileSync('sample.txt', 'utf8');
console.log(data);
console.log('This line runs AFTER the file has been fully read.');
```

Run it and compare the output order to Task 2. Take a screenshot showing both programs' output side by side.

Save it as

read-comparison.png

Task 4 – Writing To A File

Create

write-file.js

```
const fs = require('fs');
```

```
fs.writeFile('output.txt', 'Hello from Lab 06!', (err) => {
  if (err) throw err;
  console.log('File written successfully.');
```

```
});
```

Run it once, check output.txt. Run it a SECOND time with different text inside the writeFile call, and check output.txt again — note what happened to the first line you wrote.

Task 5 – Appending To A File

Create

append-file.js

```
const fs = require('fs');
```

```
fs.appendFile('output.txt', '\nThis line was appended.', (err) => {
  if (err) throw err;
  console.log('Content appended successfully.');
```

```
});
```

Run it 2–3 times and check output.txt — confirm every run adds a new line instead of replacing what's already there.

Task 6 – Deleting A File

Create

delete-file.js

```
const fs = require('fs');
```

```
fs.unlink('output.txt', (err) => {
  if (err) throw err;
  console.log('File deleted successfully.');
```

```
});
```

Run it and confirm output.txt is gone. Then run it a SECOND time immediately — read the error message you get and explain, in one sentence, why it happens.

Task 7 – Rewriting With Async/Await

Create

async-await-version.js

Use `fs.promises` instead of the regular `fs` module, and rewrite a read-then-write sequence using `async/await` and `try/catch`.

```
const fs = require('fs').promises;

async function copyContent() {
  try {
    const data = await fs.readFile('sample.txt', 'utf8');
    await fs.writeFile('copy.txt', data);
    console.log('Copied successfully.');
```

```
  } catch (err) {
    console.error('Something went wrong:', err.message);
  }
}
```

Run it and confirm `copy.txt` is created with the same content as `sample.txt`.

Task 8 – Mini Project: Command-Line Notes App

Build a tiny two-program app that stores notes in a file, using command-line arguments.

Create

add-note.js

```
const fs = require('fs');

const note = process.argv[2];

if (!note) {
  console.log('Please provide a note. Example: node add-note.js "Buy groceries"');
  process.exit(1);
}

const entry = `[${new Date().toLocaleString()}] ${note}\n`;

fs.appendFile('notes.txt', entry, (err) => {
  if (err) throw err;
  console.log('Note added!');
});
```

Create

read-notes.js

```
const fs = require('fs');

fs.readFile('notes.txt', 'utf8', (err, data) => {
  if (err) {
    console.log('No notes found yet.');
```

```
    return;
  }
  console.log(data);
});
```

Test it from the terminal:

```
node add-note.js "Submit Lab 06 by Friday"
```

```
node add-note.js "Revise the Event Loop before the mid-term"
```

```
node read-notes.js
```

Take a screenshot showing at least 2 notes added and then listed.

Save it as

Task 9 – Reflection Notes

Create a text file named

reflection-notes.txt

Answer the following in your own words (4–5 sentences each):

- Using what you observed in Tasks 2 and 3, explain why `fs.readFile()` does not block the rest of your program, while `fs.readFileSync()` does.
- In the Notes App, what could go wrong if two people ran `add-note.js` at the exact same moment, both appending to the same `notes.txt` file?

README.md

Update your `README.md` to also include:

- A one-line summary of what each `.js` file in this lab demonstrates
- Lab Number: 06
- Date

GitHub Submission

Use your existing repository — no need to create a new one.

Step 1

Open Terminal inside NodeJS-Lab.

Step 2

Run the following commands.

git add .

git commit -m "Lab 06 Submission - File System Module"

git push

Google Classroom Submission

Submit

- GitHub Repository Link
- (Optional) ZIP File

If You Face Any Problem

If you are unable to complete any task, **do not leave it incomplete without explanation.**

Create a section in your **README.md** named:

Problems Faced

Mention:

- Task Number
- Error / Issue
- What you tried to solve it
- Screenshot (if applicable)

Example

Problems Faced

Task No.: Task 6

Issue:

Running delete-file.js a second time crashed with an error instead of printing my message.

Attempted Solution:

Read the error — it said ENOENT, no such file. Understood this means the file was already deleted by the first run, so there was nothing left to delete.

Students who clearly document genuine problems will receive guidance during the next lab session.